
SAÉ 2.1&2

Premier livrable

Développement du modèle et de l'IA

1 Introduction

L'objectif de cette double SAÉ va être de développer un moteur de jeu permettant de jouer à une variante du jeu **Othello** ainsi que de créer une IA pouvant y jouer. Ce jeu va s'inscrire dans la catégorie des **jeux à somme nulle**. Le programme sera développé en Java et vous devrez respecter les principes de qualité et de développement objet, ainsi que les conventions syntaxiques de Java.

Toutes les classes s'y prêtant devront avoir des **tests unitaires** associé dans un dossier prévu à cet effet : **src/test/-java**.

Cette SAÉ va être découpée en deux livrables, chacun noté indépendamment. Le rendu de chaque livrable se fera via un dépôt GIT sur la forge dans vous devez mettre **JACQUIER Yohann** ainsi que tous vos membres en tant que manager. La qualité de GIT sera prise en compte dans la notation :

- Utilisation de branche secondaire pour les différentes fonctionnalités
- Une branche master ne contenant que les fonctionnalités pleinement abouties
- Commit par les différents membres du groupe

L'ordre des livrables sera le suivant :

1. Développement de la partie modèle et introduction à l'algorithme *Minimax*.
2. Réalisation d'une interface graphique.

Au terme de ces deux jours, vous devrez avoir les fonctionnalités suivantes :

- Possibilité de jouer au jeu en ligne de commande
- Une IA, qui, à partir d'une situation donnée, renvoie le coup qui lui semblera le plus pertinent.

Votre travail devra être déposé sur la branche **master** de votre dépôt. Le dernier commit avant le 20/05 à 22h sera celui pris en compte pour la notation.

Votre projet doit être un projet MAVEN.

2 Présentation du jeu

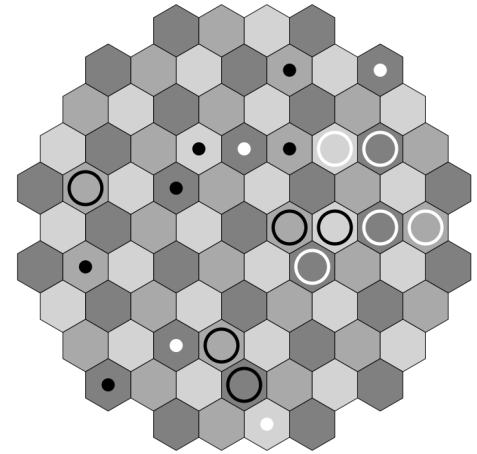
2.1 But du jeu

Le jeu va se dérouler sur un terrain Hexagonal de cette forme.

Au début de la partie, chaque joueur place où il le souhaite, **cinq anneaux** (les cercles vides sur l'image) sur le terrain. Ces anneaux pourront être déplacé selon certaines règles. En se déplaçant, les anneaux vont **créer** ou **modifier** des pions (les ronds pleins sur l'image).

L'objectif est de créer des **lignes de cinq pions**. Lorsqu'une ligne est créée, le joueur choisit un anneau et doit retirer ce dernier ainsi qu'une ligne créée. Le gagnant est le premier à réussir à enlever trois anneaux.

Si un déplacement créé plusieurs lignes, le joueur doit retirer autant d'anneaux que de lignes. Il se peut qu'un coup créé une ligne noire et une ligne blanche, dans ce cas, les deux joueurs retirent un anneau et une ligne.



2.2 Déplacement d'un anneau

Un anneau se déplace sur les 6 axes du plateau : **Nord-Ouest, Nord-Est, Est, Sud-Est, Sud-Ouest** et **Ouest**. Il peut :

- S'il n'y a aucun obstacle sur son axe, se déplacer d'autant de case qu'il le souhaite
- S'il y a un **anneau** sur son axe, il ne peut **pas** passer par dessus et n'a donc accès qu'aux cases précédentes.
- S'il y a des **pions** sur son axe, il **peut** sauter par-dessus mais doit forcément s'arrêter à la première case libre derrière le ou les pions.

Après s'être déplacé :

1. Si des pions ont été survolés, ils changent de couleur, même s'ils sont de la couleur de l'anneau
2. On place un pion de la couleur de l'anneau à son emplacement précédent

Par exemple :

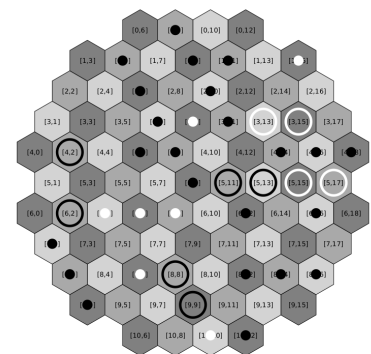
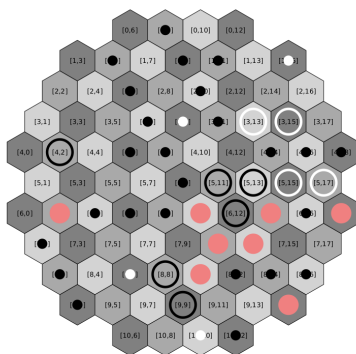


FIGURE 1 – Mouvement possible de l'anneau noir en [6,12]

FIGURE 2 – Après déplacement de l'anneau noir en [6,2]

2.3 Match nul

Si on arrive à une situation où plus aucun anneau du joueur courant ne peut bouger, la partie est considérée comme nulle, même si les deux joueurs n'ont pas le même nombre d'anneaux.

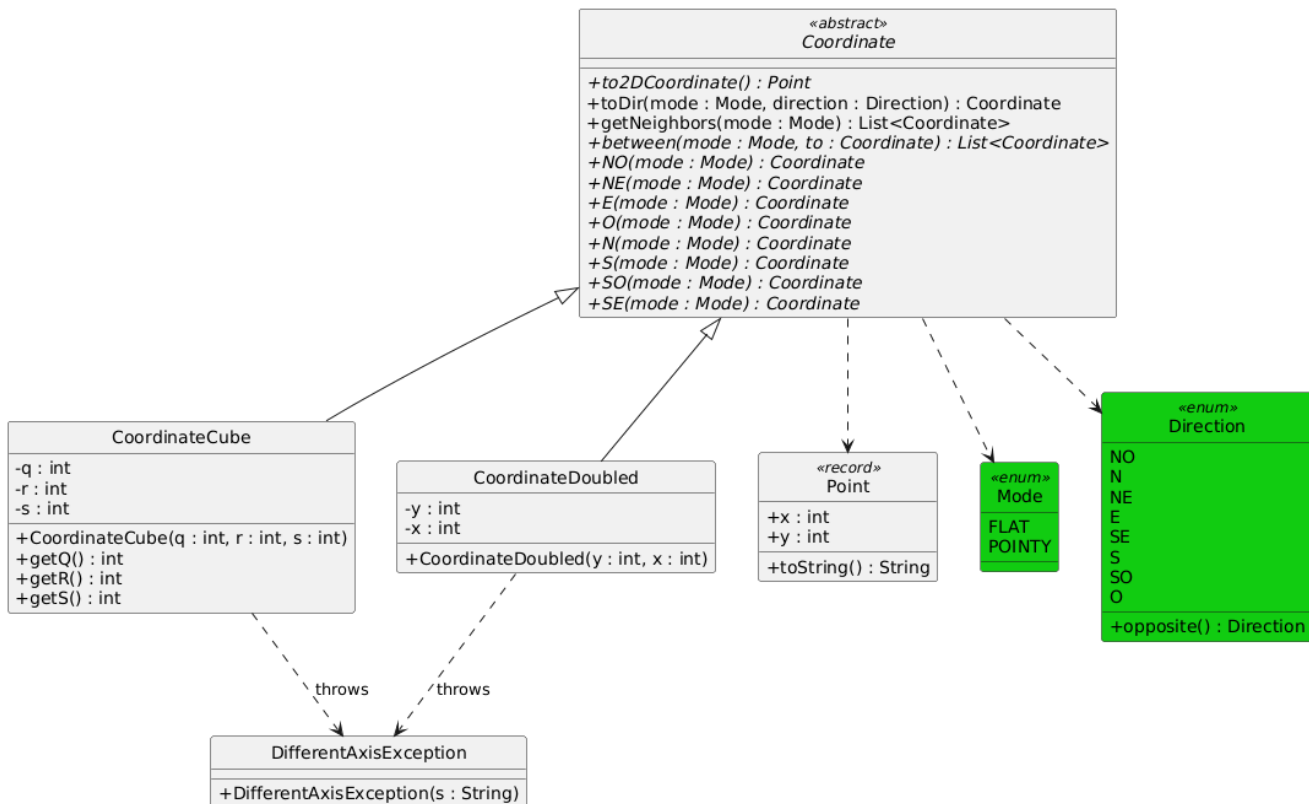
3 Modélisation des coordonnées

3.1 Module *HexagonalCoordinates*

Le principal problème avec un plateau hexagonal provient du fait qu'un tableau 2D classique ne permet pas de représenter correctement les relations de voisinage, il y en a trop peu (4 au lieu de 6), tandis qu'un tableau 3D en définit au contraire, trop (8 au lieu de 6). Il est donc nécessaire de concevoir un système de coordonnées intermédiaire, adapté à cette géométrie.

Dans une optique de réutilisation et afin de faciliter l'intégration dans de futurs projets, vous devrez créer un module (ou projet) *HexagonalCoordinate*. Ce module sera déclaré comme une **dépendance** de votre projet principal.

Pour la gestion de ce système de coordonnées, vous devez implémenter l'UML suivant :



3.2 Mode

Un terrain hexagonal peut être représenté de deux façons :

- **POINTY** : Pointe vers le haut et possède donc des voisins à gauche et à droite
- **FLAT** : Pointe vers la droite, et possède donc des voisins en haut et en bas

3.3 Coordinate

Il existe de multiples façons de représenter les coordonnées en mode hexagone. Notre jeu doit être fonctionnel indépendamment du système choisi, la classe *Coordinate* servira d'interface.

- Les fonctions **N**, **NE**, **SE**, etc. doivent lever une *InvalidParameterException* si la direction n'accepte pas le mode fourni. **Exemple** : **N** n'accepte pas le mode **POINTY**.
- La fonction *to2DCoordinate* servira à transcrire n'importe quelle coordonnée en un *Point* équivalent à une *CoordinateDoubled*.
- La fonction *between* doit lever une exception *DifferentAxisException* si deux coordonnées ne sont pas sur le même axe.
- La fonction *getNeighbors* doit **toujours** renvoyer **six** voisins. Ça sera aux autres projets de filtrer en fonction des coordonnées valide ou non. Par exemple, sur le terrain de la partie 3.4.1, la coordonnée **[0,6]** devrait quand même renvoyer 6 voisins et c'est l'application du jeu qui enlèvera les trois superflues.

3.4 CoordinateDoubled

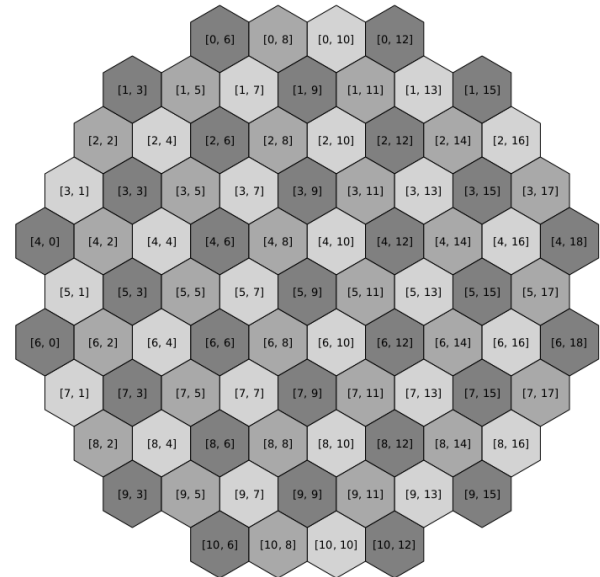
3.4.1 Le terrain

Dans un terrain représenté avec ce système, la case centrale est située en **[5,9]**.

Tous les numéros de lignes seront existant. Cependant, une ligne sur deux, les colonnes seront soit paire, soit impaire.

Sur une même ligne, les indices de colonne progressent toujours par pas de deux.

Ce terrain sera généré par la *FactoryDoubled*.



3.5 CoordinateCube

3.5.1 Un peu de définitions

Le système de coordonnées *cubique* est l'une des représentations les plus intuitives pour travailler avec un plateau hexagonal. Il repose sur trois axes (q , r , s) disposés à 120° les uns des autres. Contrairement aux systèmes classiques, ces coordonnées respectent toujours la contrainte suivante :

$$q + r + s = 0$$

Ainsi, une position n'est définie que si ses trois valeurs respectent cette équation. Cela permet de facilement vérifier si une coordonnées fournis correspond à une vraie case.

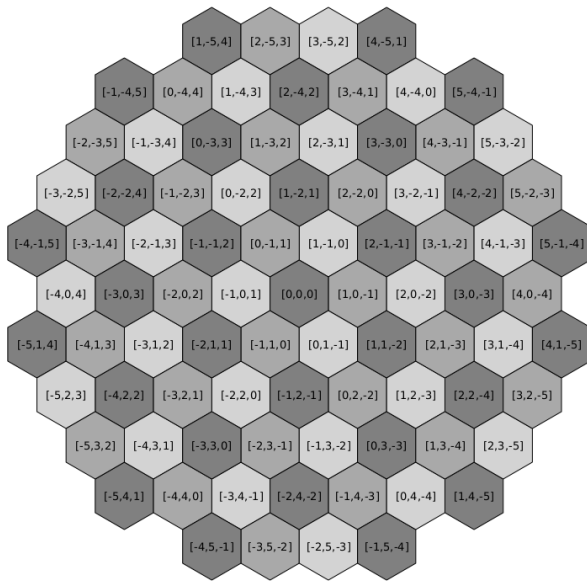
Lorsqu'un pion se déplace vers l'un des six voisins possibles, un axe augmente de 1, un autre diminue de 1 tandis que le troisième ne bouge pas.

Ce système simplifie considérablement :

- le calcul des distances entre deux cases ;
- la détermination des directions ;
- la vérification des alignements ou des axes ;
- la gestion des déplacements et des voisinages.

Le **constructeur** de cette classe doit lever une *IllegalArgumentException* si cette contrainte n'est pas remplie.

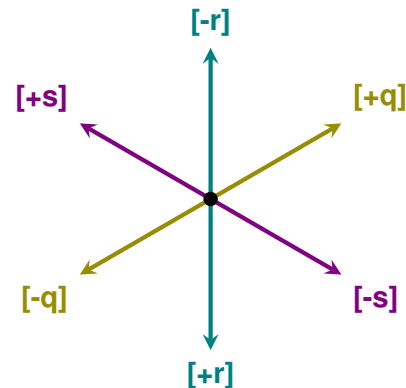
3.5.2 Le terrain



Dans un terrain représenté avec ce système, la case centrale est située en **[0,0,0]**.

Les coordonnées sont représentées dans l'ordre **[q, r, s]**, ainsi, le voisin de droite d'une coordonnée **[q, r, s]** correspond à la coordonnée **[q+1, r, s-1]**.

Pour résumer :



3.6 Pour aller plus loin

Plus d'informations et de détails sur les différents façons de gérer les coordonnées d'un espace hexagonal [ici](#).

4 Le projet principal

4.1 Gestion des dépendances

Pour commencer, ajouter votre module *HexagonalCoordinate* comme dépendance de votre projet principal.

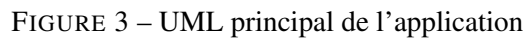
Sous **IntelliJ** :

File → **Project structure** → **Module/Votre_projet** → **Dependencies** → **+** → **Module dependency**

Sous **Eclipse** :

Clic droit sur votre projet → **Build path** → **Configure build path** → **Project** → **Add**

Votre objectif va être de recréer l'UML ci-dessous. Il n'est indiqué que les fonctions minimales demandé afin de passer nos tests unitaires, vous êtes libre d' étoffer chaque classe avec d'autres fonctions si cela vous semble utile. Les différentes sous-partie à venir donne des indications / explications sur ce qui est attendu pour les différentes classes.



Les actions du jeu n'ont pas vraiment d'interface commune : une action a besoin d'une coordonnée d'arrivée et de départ, et l'autre a besoin d'un ensemble de coordonnées à supprimer et d'une autre coordonnée unique pour l'anneau à supprimer.

4.4 *State* et *IState*

Les records sont un type de classe créé pour faciliter la définition de données fixe. Une classe record est final, on ne peut pas modifier ces attributs ni en créer en dehors du constructeur. Les getters et setters sont automatiquement créé, pas besoin de les écrire.

Chaque *State* possédera donc sa propre *HashMap* qui chacune contiendront l'ensemble des coordonnées du terrain.

Move doit lever une exception si appelée alors qu'une ligne existe sur le plateau, et inversement : *removeLine* doit lever une exception si appelée alors qu'aucune ligne n'est présente sur le plateau.

4.3.1.1 Move

Cette fonction doit créer et renvoyer le nouvel état après le déplacement de l'anneau souhaité.

Elle doit :

- Lever une *IllegalArgumentException* si la coordonnée fournit ne contient pas un anneau du joueur courant ou si la case d'arrivée n'est pas atteignable.
- Lever une *IndexOutOfBoundsException* si une coordonnée est extérieur au terrain
- Lever une *RuntimeException* si l'état courant possède une ligne à supprimer.

Si le coup joué génère des lignes pouvant être retiré par les deux équipes, on donnera la priorité à l'équipe courante pour retirer ses lignes.

4.3.1.2 RemoveLine

Cette fonction s'occupe de créer et renvoyer le nouvel état après suppression de la ligne et de l'anneau souhaité. Attention, il y peut y avoir plusieurs lignes à la fois, la suppression d'une ligne ne mène pas forcément au tour du joueur suivant.

Elle doit :

- Lever une *RuntimeException* si la ligne fournis n'est pas correct : Plus de 5 cases, n'existe pas, les cinq pions ne sont pas de la même équipe, etc.

4.3.1.3 AvailableMoves

Cette fonction renvoie la liste des positions accessible par un anneau depuis une coordonnée donnée.

4.3.1.3 Winner

La fonction *winner* renvoie *null* si aucun joueur n'a gagné, sinon elle renvoie l'équipe qui a retiré trois anneaux.

4.3.1.3 Lines

La fonction *lines* renvoie l'ensemble des lignes de 5 pions existantes sur le terrain, peu importe leur équipe. Si le terrain possède une lignes de 6 pions, *lines* devrait renvoyer une liste contenant deux *HashSet* de chacun 5 éléments.

4.3.1.4 GetPawnLines

Cette fonction **static** s'occupe de repérer toutes les lignes de 5 pions présente dans le terrain donné, peu importe leur couleur.

Bonus :

Pour aller plus vite, il est inutile de parcourir tous les pions du terrains. Seuls ceux modifiés par un mouvement peuvent créer des nouvelles lignes. Nous pourrions donc ajouter un paramètre afin de ne pas parcourir tous les pions du plateau : la liste des coordonnées qui peuvent potentiellement créer une nouvelle ligne.

4.3.1.5 RemoveToken

Fonction s'occupant de supprimer le token, s'il y en a un, à la coordonnée fournie en paramètre et renvoie le nouvel état.

4.3.1.5 ToggleToken

Cette fonction reçoit en paramètre une coordonnée, une classe de Token à créer ainsi que l'équipe de ce dernier.

La difficulté ici de créer une instance d'une classe sans écrire : **new Class(param1, param2, ...)**.

Il est possible de le faire dynamiquement dans le code, pour cela vous allez devoir :

1. Récupérer la liste des constructeurs de la classe avec la fonction **Class.getConstructors()**
2. Récupérer le premier constructeur, normalement vos classes héritant de **Token** n'en posséderont qu'un.
3. Générer une instance avec la fonction **newInstance** du constructeur récupéré.

Si le token généré est de la même classe et de la même équipe, que celui stocké à la coordonnée reçu en paramètre on le supprime du plateau, sinon, on le place.

4.4.3 State

Implémentation de la classe *IState* en se basant sur le système de coordonnée provenant du module *Hexagonal-Coordinate*.

4.5 Model

Classe s'occupant seulement de stocker l'état du jeu en cours ainsi que sa mise à jour. Les fonctions qui agissent sur l'état (tel que *move*) modifient l'état stocké dans le modèle et ne renvoie donc rien.

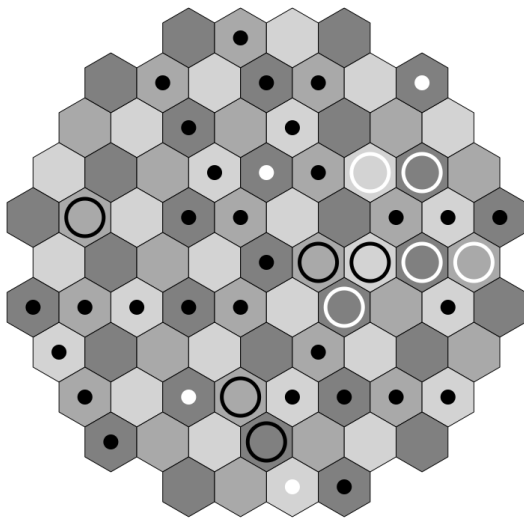
4.6 Factory

Nous allons déléguer la création d'un terrain vierge ainsi que des terrains de tests à différentes *Factory*, chacune se basant sur un système de coordonnées différentes : Une en mode *Doubled* et une en mode *Cube*.

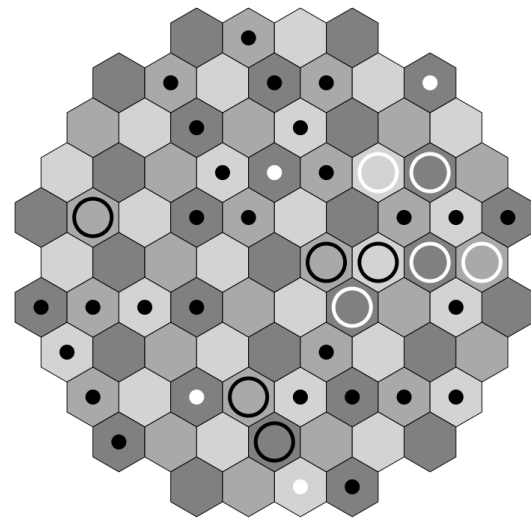
EmptyState doit s'occuper de fournir un terrain vierge, sans pion ni anneau.

Vous trouverez ci-dessous les différentes situations attendues.

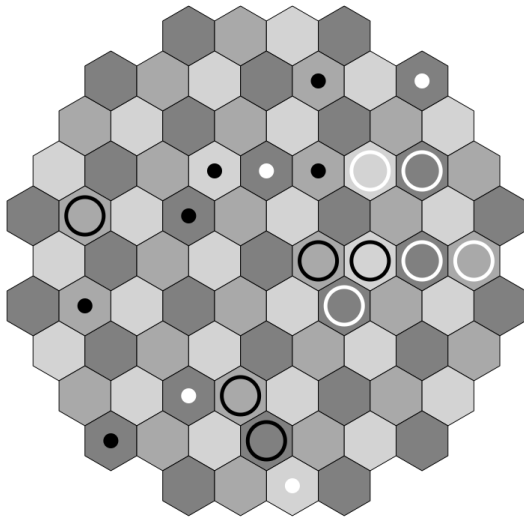
4.6.1 Les différentes situations



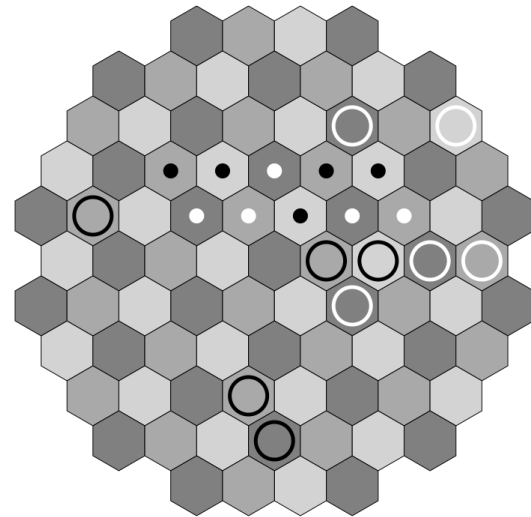
(a) stateForWhiteLineTest



(b) stateForBlackLineTest



(c) testState



(d) doubleLineStateTest

La similitude entre les différents états devra évidemment être factorisée.

Vous êtes libre de créer d'autres situations de tests si cela vous semble nécessaire.

4.7 Bonus : CUIMain

Créez une classe *CUIMain* permettant de jouer en ligne de commande. Pour pouvoir représenter le terrain, vous allez devoir vous servir de la conversion de n'importe quel coordonnée en sa version *Doubled* grâce à la fonction *to2DCoordinate*.

Pour jouer, vous générerez un état avec 5 anneaux de chaque équipe placé aléatoirement.

Une fois cela fait, vous créez un tableau 2D assez grand pour accueillir tous les éléments du terrain et vous les placez au bon endroit.

Il n'y a plus qu'à afficher le terrain. Une représentation possible serait :

```

      _ . _ _
     _ . _ . . _ .
    _ _ . _ x _ _ _
   _ _ _ . . . O O _
  _ O _ . x _ _ . . .
 _ _ _ _ . o o o O
. . x x x _ O _ . _
. _ _ _ _ . _ _ _
. _ . o . . . .
. _ _ o _ _ _
_ _ . .

```

Avec :

- o pour les anneaux blancs
- O pour les anneaux noirs
- _ pour une case vide
- . pour un pion blanc
- x pour un pion noir

4.7.1 Bonus : Initialisation d'une partie

Faites en sorte d'avoir une interface permettant aux différents joueurs de choisir l'emplacement de leurs anneaux en début de partie.

5 Développement de l'IA

L'objectif de cette partie va être de créer une IA qui, en exploitant les parcours de graph que vous avez étudié en Mathématiques, fournira pour une situation donnée le coup lui semblant le plus pertinent.

L'algorithme [Minimax](#) que vous allez mettre en place s'appuie sur un parcours en profondeur.

5.1 Classe Node

Vous allez commencer par créer une classe *Node* qui permettra de modéliser un noeud du graph, soit une situation de jeu.

Cette classe devra donc stocker plusieurs choses :

1. L'état du jeu représenté par ce noeud : Position des anneaux et des pions ;
2. Le noeud parent qui a permis d'atteindre ce noeud ;
3. L'action qui a permis de rejoindre ce noeud : Soit un déplacement d'anneau, soit une suppression de ligne.

5.2 Algorithme MiniMax

Cette partie décrit seulement le fonctionnement de l'algorithme, le travail demandé sera situé en **5.3**.

5.2.1 Principe du minimax

L'algorithme explore l'arbre des états du jeu jusqu'à une profondeur d . Minimax va attribuer un score à chaque noeud permettant d'estimer s'il est intéressant ou non pour le joueur d'atteindre cette situation. A chaque changement d'étage, on tente soit de maximiser le score (Quel est le meilleur coup que l'on peut faire ?) soit de le minimiser (Si l'adversaire joue parfaitement, à quel point peut-il nous mettre en difficulté ?).

L'algorithme va d'abord aller évaluer les feuilles de son arbre avant de remonter progressivement et d'attribuer une note à tous les noeuds en *bottom-up*.

Voici un exemple où $d = 2$:

- **Nœud Max (L'IA)** : Ce sont les situations où c'est à l'IA de jouer, elle va donc prendre l'enfant qui crée la situation la plus avantageuse pour elle.
- **Nœud Min (Adversaire)** : Cette fois ce sont les situations où c'est à l'adversaire de jouer, on part donc du principe que celui-ci choisira le coup menant à la situation qui nous avantage le moins.

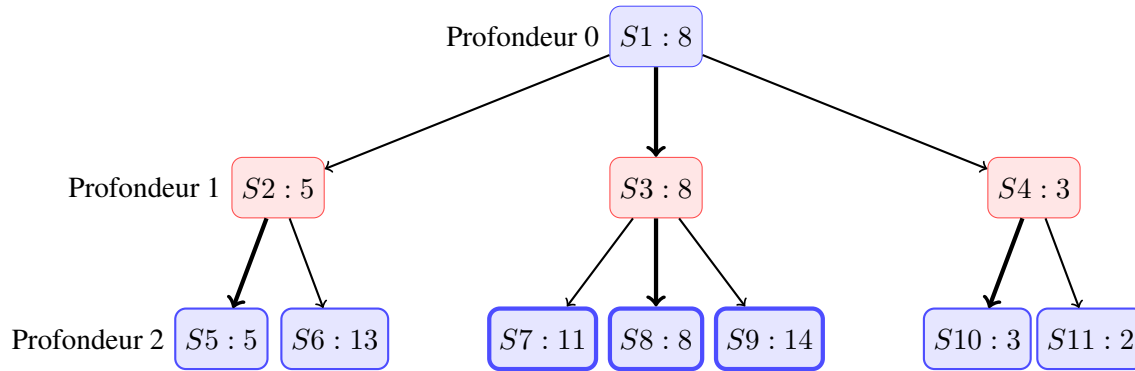


FIGURE 5 – Evaluation d'une situation

Dans l'ordre :

1. La situation initiale $S1$ est développée, générant trois successeurs directs : $S2$, $S3$ et $S4$.
2. On commence par explorer $S2$: les situations $S5$ et $S6$ sont générées puis évaluées (leur profondeur étant égale au d choisi) et valent respectivement 5 et 13. La valeur minimale étant 5, le nœud $S2$ hérite par extension de cette évaluation ($S2 = 5$).
3. Les situations $S3$ et $S4$ sont évaluées après exploration de leurs propres descendants. Leurs valeurs respectives sont fixées à 8 et 3.
4. Une fois l'ensemble des options explorées, $S1$ compare les résultats. La situation $S3$ présentant la valeur la plus élevée, elle est sélectionnée comme la stratégie optimale pour maximiser les chances de victoire et on choisit donc l'action qui nous y mène.

5.2.2 Fonction d'évaluation

Maintenant que l'on sait comment choisir une situation en fonction du score de ces enfants, on va mettre en place la fonction permettant d'évaluer les feuilles d'un de ces graph. La qualité de cette fonction déterminera à 90% la qualité de votre IA.

Une idée d'évaluation est de faire la somme de différents facteurs, chacun pondéré selon un poids qui est lié à son importance. Une possibilité de poids et de facteurs pourrait être :

Critère	Poids	Description
Victoire / Défaite	$\pm 100\,000$	Un joueur a retiré 3 anneaux.
Anneau en moins	$\pm 1\,000$	Progrès direct vers la victoire.
Ligne de 4 pions	± 50	Menace immédiate de retrait d'anneau.
Mobilité	$\pm 0,5$	Nombre de déplacements possibles pour les anneaux.

TABLE 1 – Heuristique d'évaluation des situations

A vous d'imaginer et de tester différents facteurs / poids voir quels choix rendent votre IA plus performante.

Il y a bien sur d'autres critères qui peuvent être pris en compte, comme notamment la place des pions sur le plateau : un pion centrale a plus de valeur qu'un pion sur les bords. Une idée serait de créer un masque permettant de donner un poids à chaque case afin de facilement pouvoir calculer cela.

5.2.3 Élagage Alpha-Beta

Pour éviter au maximum d'explorer des branches inutiles, on peut utiliser deux variables :

- α : La meilleure valeur que le joueur **Max** peut garantir à ce moment de l'exploration.
- β : La meilleure valeur que le joueur **Min** peut garantir.

Dès que $\alpha \geq \beta$, on peut arrêter d'explorer cette branche. On sait que notre adversaire nous forcera à avoir le score β à partir de celle-ci, alors qu'un autre coup nous garanti le score α que nous allons donc privilégier.

5.3 Travail à réaliser

Créer une interface *AI* contenant une fonction **Action chooseMove(IState state)**.

Créer une classe *MinimaxAI* implémentant l'interface *AI* et l'algorithme Minimax décrit dans la partie précédente.

Créer une classe de jeu *MainAI* permettant de jouer contre votre IA.